

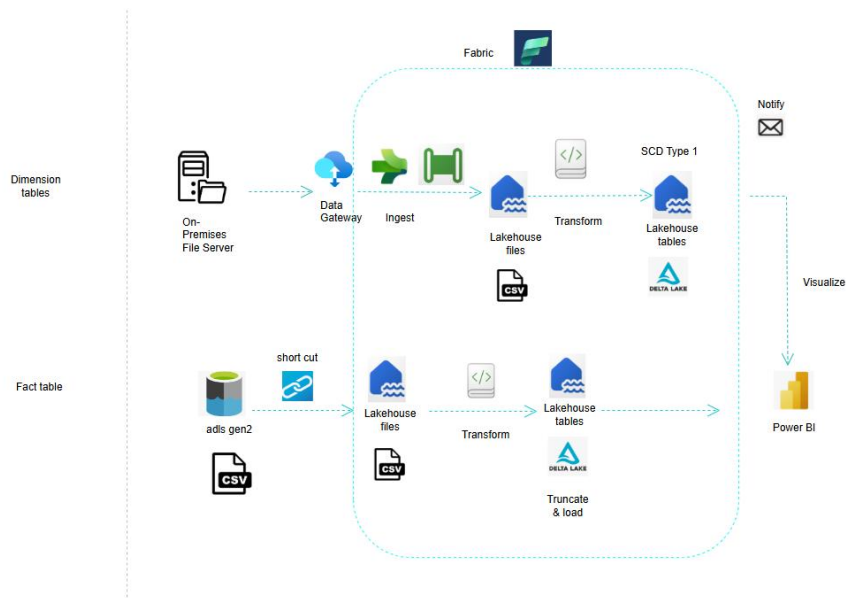
End-to-End Data Engineering & Analytics Documentation:

Project Fabric_Financial_Analytics

This document outlines the architecture, implementation steps, and best practices for the end-to-end bank data analytics pipeline using **Microsoft Fabric**. The pipeline ingests on-premises and cloud-hosted CSV files, applies cleansing and dimension mapping transformations, builds a star-schema semantic model, and exposes the data for Power BI reporting.

Architecture Overview

The architecture utilizes a **Medallion Architecture** (Bronze -> Silver -> Gold) to ensure data quality and decoupled storage.



Data Pipeline Components

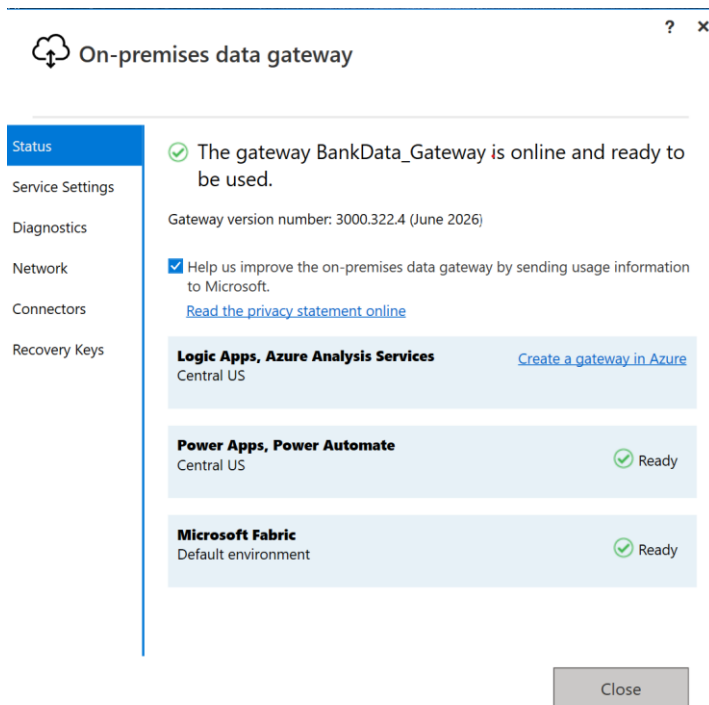
- Bronze Layer (Raw Files):**
On-premises Dimension CSVs landed into Lakehouse /Files via Data Gateway.
Cloud Fact CSV surfaced in Lakehouse /Files via an ADLS Gen2 Shortcut.
- Silver Layer (Staging Tables):** Raw CSVs converted to initial Delta tables with strict schema enforcement.
- Gold Layer (Final Tables):** Cleansed and transformed tables containing production-ready Star-Schema dimensions and facts.
- Semantic / Reporting Layer:** Direct Lake custom semantic model driving high-performance Power BI visuals.

Phase 1: Ingestion & Connectivity

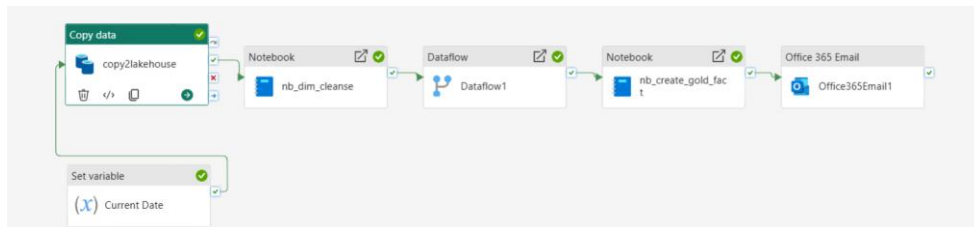
1. Ingesting On-Premises Dimensions via Data Gateway

To bring the local dimension CSV files into the Fabric Lakehouse:

- Install and Configure:** On-Premises Data Gateway is installed and registered on local network/server.



2. **Create a Data Pipeline:** In Fabric Workspace, click + New -> **Data pipeline** and name it pl_ingest_data_to_lh.



3. **Add Copy Data Activity:** Drag a **Copy data** activity onto the canvas.
4. **Configure Source:**
 - **Connection:** Create a new connection selecting **File system**
 - **Gateway:** Select registered On-Premises Data Gateway.
 - **File Path:** Provide the local directory path to dimension CSV files (e.g., C:\BankData\Dimensions\).

General	Source	Destination	Mapping	Settings
Connection * onpremconnection Refresh Test connection Edit				
File path type ☒ File path ☐ File filter ☐ Wildcard file path ☐ List of files ⓘ				
File path files / File name Browse Preview data				
Recursively ⓘ ☒				
File format * DelimitedText Settings				
> Advanced				

5. Configure Destination:

- **Target Data Store:** Select **Workspace**.
- **Workspace / Workspace Item:** Select your current workspace and target **Lakehouse**.
- **Root Folder:** Choose **Files**.
- **Directory:** Set to bankdata-dim/.

General Source **Destination** Mapping Settings

Connection * lh_fabric (fabricws) Refresh Open

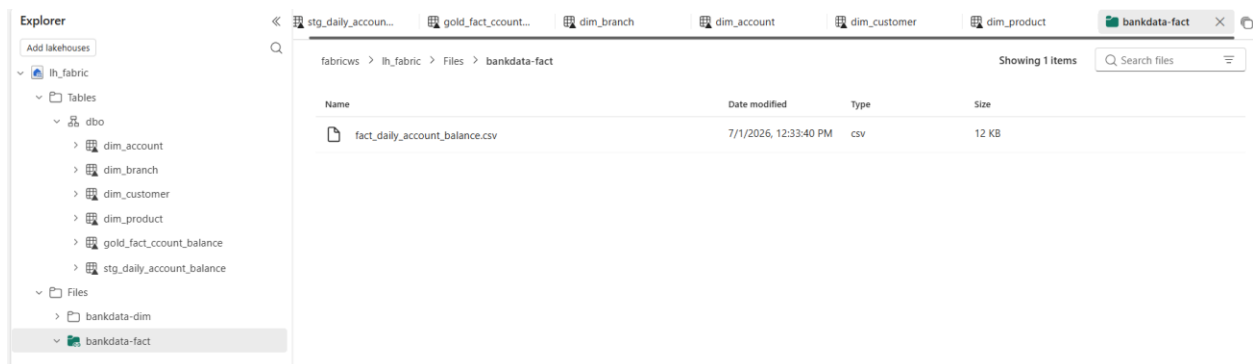
Root folder Tables ☒ Files

File path bankdata-dim / File name Browse Preview data

File format * DelimitedText Settings

> Advanced

2. Connecting Cloud Fact Data via ADLS Gen2 Shortcut



The fact CSV resides in Azure Data Lake Storage Gen2 and must be referenced without moving the underlying data.

1. Open your target **Lakehouse** in Fabric.
2. In the left Explorer pane, hover over **Files**, click the ellipsis (...), and select **New shortcut**.
3. Select **Azure Data Lake Storage Gen2**.
4. **Connection Settings:**
 - **URL:** Use your endpoint root: https://<your_storage_account_name>.dfs.core.windows.net/
 - **Authentication Kind:** Select **Workspace Identity** or *Organizational Account*.
 - *Note: Ensure Fabric Workspace Identity has been granted the **Storage Blob Data Contributor** role on the storage account in the Azure Portal.*
5. **Shortcut Details:**
 - **Shortcut Name:** Set as bankdata-fact.
 - **Sub path:** Browse to the container and folder where the fact CSV file resides. Click **Create**.

Phase 2: Staging & Transformation (Dataflow Gen2)

Once raw files are visible in the /Files section, use **Dataflow Gen2** to clean the records, evaluate mathematical constraints, and establish staging Delta tables.

The screenshot displays the Power Query Editor for a query named 'fact_daily_account_balanc...'. The 'Applied steps' list includes Source, Navigation, Navigation 1, Navigation 2, Navigation 3, Navigation 4, Imported CSV, Promoted headers, Changed column..., Marked key column..., Added custom, Filtered rows, and Removed duplicate... The data table shows 11 rows of transaction data. The formula bar contains the following MEX function:

```
Table.SelectRows(8'Added custom', each [product_key] <> null and [product_key] <> null and [branch_key] <> null and [branch_key] <> null and [customer_key] <> null and [customer_key] <> null and [account_key] <> null and [account_key] <> null and [date_key] <> null and [date_key] <> null and [Is_Balance_Valid] = true)
```

1. Initialize the Dataflow

1. In your workspace, click **+ New** -> **Dataflow Gen2**. Name it `df_cleanse_fact_table`.
2. Click **Get data** -> **Lakehouse** -> navigate to your workspace and open your Lakehouse's **Files** folder.
3. Select the dimension files and your fact shortcut file. Click **Transform Data**.

2. Apply Cleansing Logic

Execute the following modifications within the Power Query canvas:

- **Promote Headers:** Select **Use first row as headers** on the Home tab.
- **Fix Data Types:** Explicitly set numerical text keys to **Whole number**, dollar values to **Decimal number**, and dates to **Date** or **Date/Time** types.
- **Trim and Clean Text:** Right-click text column headers -> **Transform** -> **Trim** to remove trailing whitespace.
- **Handle Nulls/Blanks:** Drop down on primary key columns and uncheck **Null** to drop invalid rows.

3. Add Mathematical Balancing Clause

To filter out transaction rows that contain data anomalies, apply an accounting validation step:

1. Navigate to the **Add column** tab -> click **Custom column**.
2. **Column Name:** `Is_Balance_Valid`
3. **Formula:** Enter the mathematical equivalence statement:

$$[\text{Closing Balance}] = [\text{Opening Balance}] + [\text{Credit}] - [\text{Debit}]$$
4. Click **OK**.
5. Filter the newly generated column: click the filter dropdown header, **uncheck FALSE**, and click **OK**.
6. Right-click the `Is_Balance_Valid` column and select **Remove** to keep your final output schema clean.

4. Target Staging Destinations (Delta Tables)

1. For each processed query, click the + icon next to **Data destination** in the bottom-right properties pane.
2. Select **Lakehouse**, point to your workspace destination, and select **New table**.
3. Name target table as : stg_daily_account_balance
4. Ensure the update method is set to **Replace** for staging execution, then click **Publish**.

Phase 3: PySpark Notebook Transformation Layer (nb_dim_cleanse)

The PySpark processing layer acts as the transition between the Silver (Staging) and Gold (Production) zones. It enforces business logic constraints, handles null value imputation, strips whitespaces, and de-duplicates records on the natural business keys to ensure strict **SCD Type 1 (Overwrite)** data integrity for dimension tables. Add this as a notebook activity in pipeline next to copy data activity created earlier.

```
1 from pyspark.sql import functions as F
2 from pyspark.sql.types import IntegerType, StringType, DecimalType
3 from delta.tables import DeltaTable
4
5 def remove_duplicates(df, primary_key):
6     return df.dropDuplicates([primary_key])
7
8 def remove_nulls(df, critical_columns):
9     return df.dropna(subset=critical_columns)
10
11 def fill_nulls(df, default_values):
12     return df.fillna(default_values)
13
14 def load_data(format_type, file_name):
15     base_path = "abfss://fabricws@onelake.dfs.fabric.microsoft.com/lh_fabric.Lakehouse/Files/bankdata-dim"
16     return (spark.read.format(format_type)
17             .option("header", "true")
18             .option("inferSchema", "true")
19             .load(f"{base_path}/{file_name}.csv"))
20
```

```
def apply_schema_and_casting(df, casting_rules):
    """
    Dynamically applies casting, trimming, upper-casing, AND renames
    the column from source name to target name.
    """
    select_exprs = []
    for target_col, rule in casting_rules.items():
        # Look for the column using its source name, default to target name if not provided
        source_col = rule.get("source_col_name", target_col)
        expr = F.col(source_col)

        # Apply string cleanups if specified
        if rule.get("trim"):
            expr = F.trim(expr)
        if rule.get("upper"):
            expr = F.upper(expr)

        # Apply data type casting
        if "type" in rule:
            if rule["type"] == "date" and "format" in rule:
                expr = F.to_date(expr, rule["format"])
            else:
                expr = expr.cast(rule["type"])

        # ALIAS directly to the target table's column name
        select_exprs.append(expr.alias(target_col))

    return df.select(*select_exprs)
```

Phase 3: Gold Layer Joins & Referential Integrity (PySpark)

To handle unmatched foreign keys gracefully before saving to your production reporting layer, execute a PySpark notebook to apply a Left Join and fill missing values with -1.

1. Create a **New Notebook** inside your Fabric workspace and attach it to your Lakehouse; include this as a notebook activity (nb_create_gold_fact) next to dataflow activity created for fact table cleansing in the pipeline.
2. Paste and execute the following PySpark routine:

Testing my code syntax

```

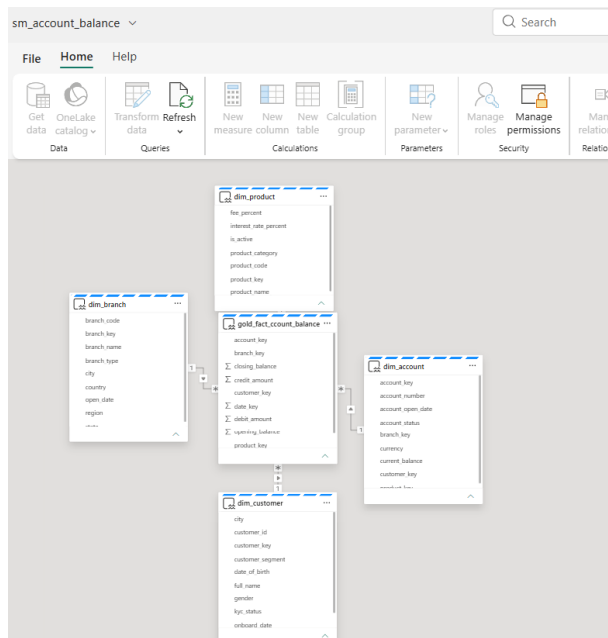
8 dim_customer_df = spark.read.table("dim_customer")
9 dim_account_df = spark.read.table("dim_account")
10 dim_branch_df = spark.read.table("dim_branch")
11 dim_product_df = spark.read.table("dim_product")
12
13 # 2. Perform Left Joins and handle missing keys using F.coalesce
14 # F.coalesce takes the first non-null value it finds. If the dim key is null, it falls back to -1.
15 final_fact_df = staged_fact_df \
16     .join(dim_customer_df, staged_fact_df["customer_key"] == dim_customer_df["customer_key"], "left") \
17     .join(dim_account_df, staged_fact_df["account_key"] == dim_account_df["account_key"], "left") \
18     .join(dim_branch_df, staged_fact_df["branch_key"] == dim_branch_df["branch_key"], "left") \
19     .join(dim_product_df, staged_fact_df["product_key"] == dim_product_df["product_key"], "left") \
20     .select(
21         staged_fact_df["date_key"],
22         F.coalesce(dim_branch_df["branch_key"], F.lit(-1)).alias("branch_key"),
23         F.coalesce(dim_product_df["product_key"], F.lit(-1)).alias("product_key"),
24         # If the dimension join fails, the dim key is null -> replaced with -1
25         F.coalesce(dim_customer_df["customer_key"], F.lit(-1)).alias("customer_key"),
26         F.coalesce(dim_account_df["account_key"], F.lit(-1)).alias("account_key"),
27         staged_fact_df["opening_balance"],
28         staged_fact_df["debit_amount"],
29         staged_fact_df["credit_amount"],
30         staged_fact_df["closing_balance"],
31         staged_fact_df["transaction_count"]
32     )
33
34 # 3. Save the treated data into the final Delta table
35 # Use 'append' or 'overwrite' depending on your pipeline logic
36 final_fact_df.write \
37     .format("delta") \
38     .mode("overwrite") \
39     .saveAsTable("gold_fact_ccount_balance")

```

Phase 4: Modeling & Reporting (Power BI)

1. Build a Custom Semantic Model

1. Return to workspace list and click + New -> **Semantic model**. Name it sm_account_balance.



2. Add your definitive production tables: gold_fact_account_balance, dim_customer, dim_branch, dim_product and dim_account. Click **Confirm**.
3. In the modeling ribbon, click **Open data model**.
4. **Establish Links (Relationships):**
 - Drag customer_key from gold_fact_account_balance and drop it onto customer_key in dim_customer. Like wise add mapping on account_key, branch_key and product_key.
 - Set Cardinality to **Many to one (*:1)** and Cross filter direction to **Single**. Click **Save**.

2. Write Required Business Analytics Measures

Right-click your fact table block inside the data model space, select **New measure**, and instantiate your reporting metrics using DAX:

- **Distinct Accounts Volume:**

Code

Distinct Accounts Count = DISTINCTCOUNT(gold_fact_account_balance [account_number])

- **Total Balance Outstandings:**

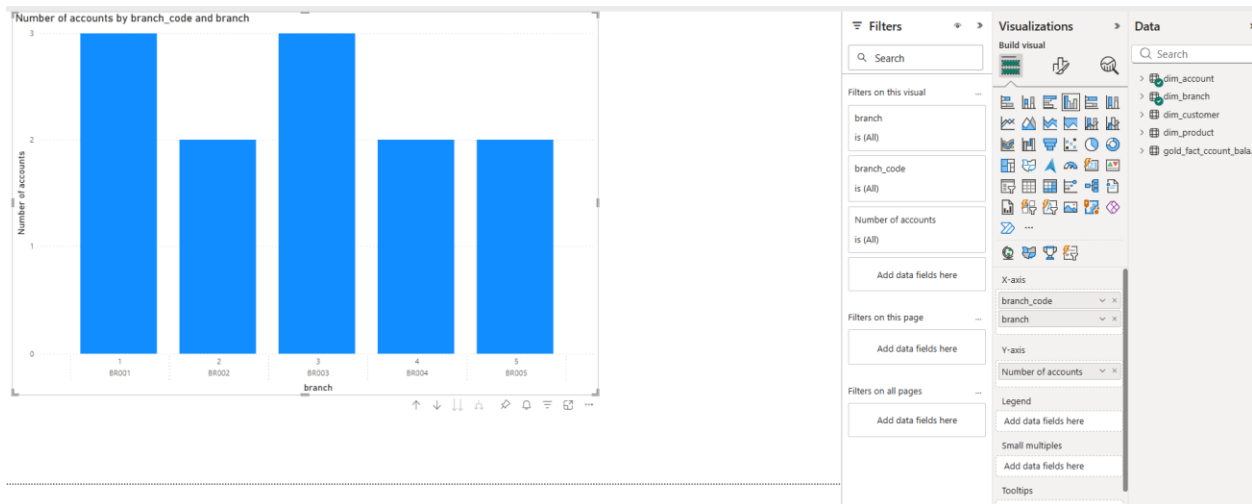
Code

Total Balance = SUM(gold_fact_account_balance[current_balance])

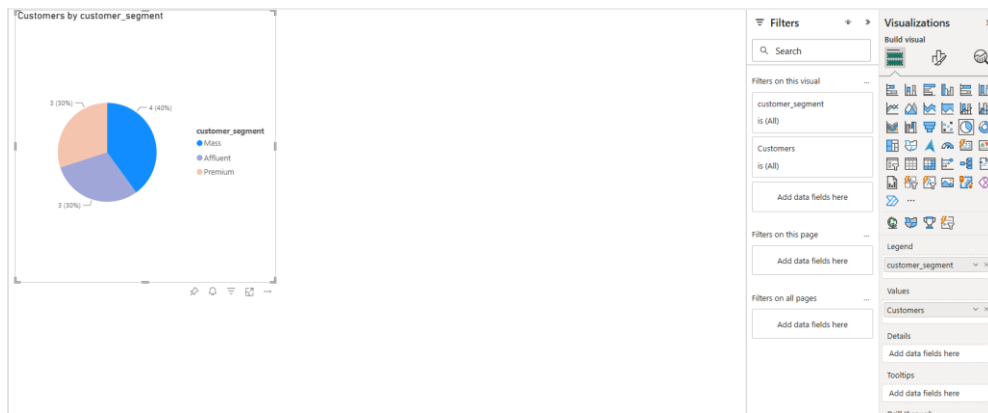
3. Visualizations Configuration

Click **New report** from the top ribbon to enter the Power BI design canvas.

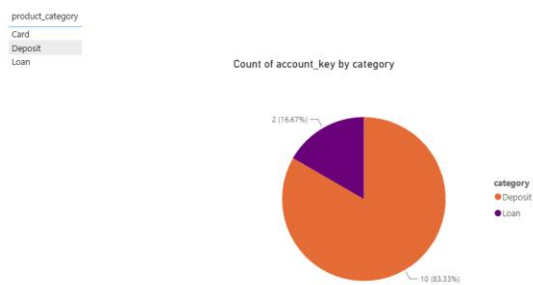
Component A: Distinct Accounts Breakdown by Branch



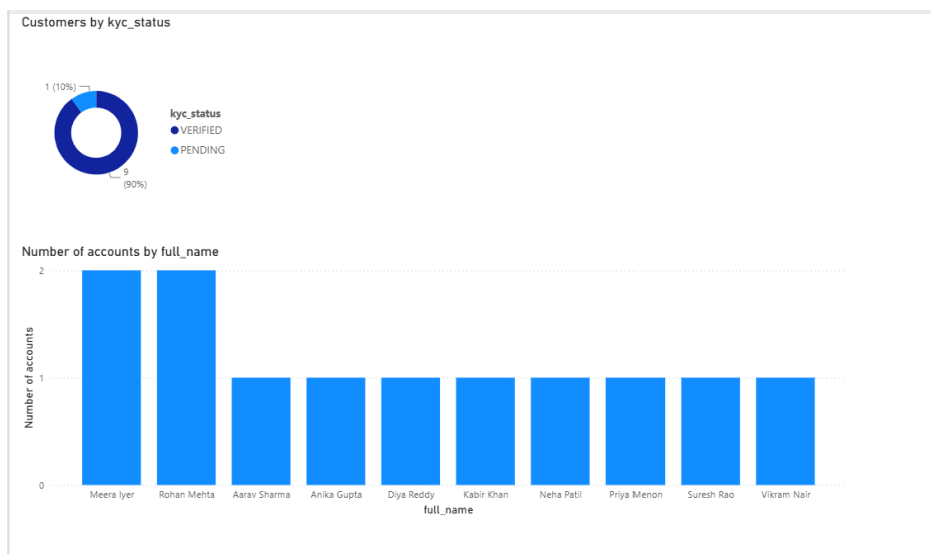
Component B: Customers breakdown by customer_segment



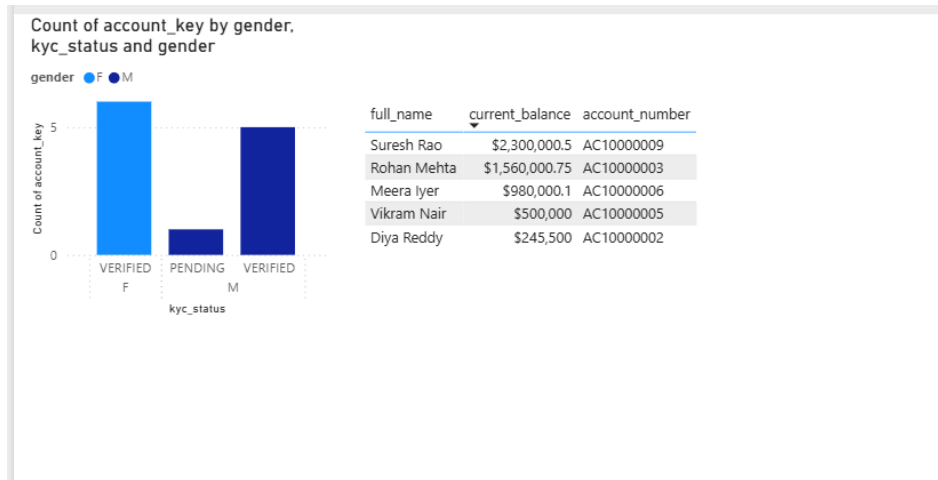
Component C: Accounts breakdown by product category; Customers breakdown by kyc_status



Accounts per customer



Component D: Accounts count by gender and top 5 customers and their balances



Phase 5: Operations & Automated Notifications

To ensure stakeholders are notified if data processing runs into failures or success, add an email activity in pipeline

1. Sequence the Master Pipeline

1. Open a master control pipeline.
2. Go to the **Activities** ribbon tab and drag out an **Office 365 Outlook** activity block. Name it Send success notification.

2. Configure success notifications

1. Draw a green **Arrow (On failure)** from both the Dataflow and Notebook activity boxes and hook them directly into the input side of the Outlook activity block.
2. Select the Outlook activity box and open its bottom **Settings** panel:
 - **To:** Provide production support emails separated by semicolons.
 - **Subject:** NOTIFICATION: Fabric Bank Pipeline Ingestion Success
 - **Body:** Click **Add dynamic content** and use the following structural text message:

Plaintext

@concat(

'Team, \n\nThe bank data processing pipeline succeeded.',

' \n\nWorkspace Pipeline Run ID: ', pipeline().RunId,

' \n\nTimestamp: ', utcnow()

)

3. Schedule the Refresh

Go to your master data pipeline's settings page in the workspace view, toggle **Schedule** to **On**, and assign your preferred daily refresh interval (e.g., every morning at 6:00 AM) to automatically run your data processing completely hands-free.

Summary

The **Project BankData Fabric** platform establishes a modernized, automated, end-to-end data processing framework. It successfully unifies disparate corporate data assets—ranging from legacy on-premises dimension files to cloud-hosted transactional fact files—into a high-performance, single source of truth within Microsoft Fabric **OneLake**.

Key Business & Technical Outcomes:

- **Decoupled Architecture:** Utilizes a standard **Medallion Architecture** to isolate raw file ingestion (Bronze) from cleansed data processing tables (Silver) and production-ready analytical star-schema datasets (Gold).
- **Zero-Copy Cloud Integration:** Leverages **OneLake Shortcuts** to directly reference massive transactional fact files in Azure Data Lake Storage (ADLS) Gen2 without incurring additional data duplication or egress fees.
- **Strict Referential Integrity:** Implements a programmatic PySpark transformation layer to perform robust outer joins, automatically assigning a default surrogate key of -1 to unmapped or orphan transaction records, preventing downstream dashboard errors.
- **Direct Lake Performance:** Connects Power BI via the native **Direct Lake Mode**, granting stakeholders blazing-fast data-load speeds directly from OneLake Parquet files without the need for traditional import schedules or restrictive DirectQuery latency.

2. Technical Issues Log & Resolution Register

During the development and deployment phases of this data platform, several infrastructure, network, and development bugs may arise. Use this register to quickly identify symptoms and apply tested resolutions.

Issue ID	Component	Error Message / Symptom	Root Cause	Resolution Strategy
ERR-001	Connectivity / Shortcut	Connection of kind AzureDataLakeStorage using AuthKind WorkspaceIdentity did not have accessToken specified.	The required Workspace Identity service principal does not exist or hasn't been initialized inside the Fabric workspace settings.	Navigate to Workspace Settings -> Workspace Identity and explicitly create/enable the identity. Ensure this is done by a Workspace Admin outside of "My Workspace".
ERR-002	Azure Storage / IAM	Error 403 - This request is not authorized to perform this operation using this permission.	The identity (Workspace or Organizational account) lacks data-plane Access Control (IAM) permissions on the ADLS Gen2 account.	In the Azure Portal, navigate to the storage account's Access Control (IAM) and assign the Storage Blob Data Reader or Storage Blob Data Contributor role to your Fabric workspace identity.
ERR-003	Storage Networking	Shortcut fails to connect or returns timeout errors when the storage firewall is enabled.	The Azure Storage account restricts public network access and is blocking Fabric's internal backend compute engines.	In the Storage Account under Networking, check "Allow trusted Azure services to access this storage account" and configure a specific Resource Instance Rule targeting Microsoft.DataFactory/workspaces.

			The Office 365 Outlook activity requires a cloud-hosted Exchange Online mailbox to invoke Microsoft Graph REST APIs, which fails on hybrid or local mailboxes.	
ERR-004	Pipeline Notifications	<pre>{"error":{"code":"MailboxNotEnabledForRESTAPI","message":"The mailbox is either inactive... or hosted on-premise."}}</pre>	The Office 365 Outlook activity requires a cloud-hosted Exchange Online mailbox to invoke Microsoft Graph REST APIs, which fails on hybrid or local mailboxes.	Migrate the service account mailbox to Exchange Online, assign an active Exchange license, or swap the notification activity to use the Microsoft Teams pipeline activity block instead.